

Unidad 11: Playground Intermedio

Parte II

Herencia de Templates

La herencia de templates es una técnica en Django que permite reutilizar código HTML para mantener una estructura consistente en varias páginas web. Este método se basa en definir un template "padre" que contiene elementos comunes a todas las páginas (como la barra de navegación, el pie de página, y el logo), y luego extender ese template en los templates "hijos" que heredan y pueden modificar su contenido específico.

¿Cómo definir un template "padre"?

1. Crear el Template Padre:

- El primer paso es crear un archivo HTML que actuará como el template base. Este archivo incluirá las partes de la página web que serán comunes a todas las páginas.
- Ejemplo de `padre.html`:

```
``html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}My Site{% endblock %}</title>
</head>
<body>
  <header>
    <nav>
      <ul>
        <li><a href="{% url 'inicio' %}">Inicio</a></li>
        <li><a href="{% url 'cursos' %}">Cursos</a></li>
        <li><a href="{% url 'contacto' %}">Contacto</a></li>
      </ul>
    </nav>
  </header>
  <main>
    {% block content %}
    <!-- Contenido específico de cada página irá aquí -->
    {% endblock %}
  </main>
```

```
<footer>
  <p>&copy; 2024 My Site</p>
</footer>
</body>
</html>

...

```

2. Definir Bloques:

- Dentro del template padre, se definen bloques usando la sintaxis `{% block nombre\_del\_bloque %}{% endblock %}`. Estos bloques pueden ser reemplazados o extendidos en los templates hijos.
- En el ejemplo anterior, `title` y `content` son bloques que los templates hijos pueden modificar.

¿Cómo extender un template "padre" en un template "hijo"?

1. Crear el Template Hijo:

- Los templates hijos extienden el template padre usando la directiva `{% extends 'ruta/al/template/padre.html' %}`.
- Luego, definen el contenido específico para los bloques que se desea modificar.
- Ejemplo de `hijo.html`:

```
``html
{% extends 'padre.html' %}

{% block title %}Página de Cursos{% endblock %}

{% block content %}
  <h1>Bienvenido a la página de cursos</h1>
  <p>Aquí encontrarás todos los cursos disponibles.</p>
{% endblock %}

...

```

Beneficios de la Herencia de Templates:

Consistencia:

Asegura que todos los templates compartan un diseño común, lo que facilita la actualización y el mantenimiento del sitio web.

Reutilización de Código:

Evita la duplicación de código, ya que los elementos comunes se definen una vez en el template padre.

Flexibilidad:

Permite que cada página web tenga contenido específico mientras mantiene la estructura general del sitio.

Ejemplo Completo:

1. Template Padre (`padre.html`):

```
``html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}My Site{% endblock %}</title>
</head>
<body>
  <header>
    <nav>
      <ul>
        <li><a href="{% url 'inicio' %}">Inicio</a></li>
        <li><a href="{% url 'cursos' %}">Cursos</a></li>
        <li><a href="{% url 'contacto' %}">Contacto</a></li>
      </ul>
    </nav>
  </header>
  <main>
    {% block content %}
    <!-- Contenido específico de cada página irá aquí -->
    {% endblock %}
  </main>
  <footer>
```

```
<p>&copy; 2024 My Site</p>
</footer>
</body>
</html>
```

...

2. Template Hijo (`hijo.html`):

```
``html
{% extends 'padre.html' %}

{% block title %}Página de Cursos{% endblock %}

{% block content %}
<h1>Bienvenido a la página de cursos</h1>
<p>Aquí encontrarás todos los cursos disponibles.</p>
{% endblock %}
``
```

La herencia de templates en Django es una herramienta poderosa que simplifica la gestión y el mantenimiento de sitios web, permitiendo una estructura coherente y un diseño uniforme en todas las páginas.

Navegar entre Templates

Creación del Menú de Navegación en el Template Padre

1. Definir el Template Padre:

- El primer paso es crear un archivo HTML que actuará como el template base para nuestro sitio web. Este archivo incluirá el menú de navegación que será común a todas las páginas.
- Ejemplo de `padre.html`:

```
``html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}My Site{% endblock %}</title>
  <link rel="stylesheet" href="{% static 'css/styles.css' %}">
</head>
<body>
```

```

<header>
  <nav class="navbar navbar-light bg-light static-top">
    <div class="container">
      <a class="navbar-brand" href="{% url 'inicio' %}">Inicio</a>
      <a class="navbar-brand" href="{% url 'cursos' %}">Cursos</a>
      <a class="navbar-brand" href="{% url 'profesores' %}">Profesores</a>
      <a class="navbar-brand" href="{% url 'estudiantes' %}">Estudiantes</a>
      <a class="navbar-brand" href="{% url 'entregables' %}">Entregables</a>
      <a class="btn btn-primary" href="#NADAAUN">INICIAR</a>
    </div>
  </nav>
</header>
<main>
  {% block content %}
  <!-- Contenido específico de cada página irá aquí -->
  {% endblock %}
</main>
<footer>
  <p>&copy; 2024 My Site</p>
</footer>
</body>
</html>

```

...

Ajuste de los Templates Hijos para Utilizar el Menú

1. Extender el Template Padre:

- Los templates hijos extienden el template padre usando la directiva `{% extends 'ruta/al/template/padre.html' %}`.
- Luego, definen el contenido específico para los bloques que se desea modificar.
- Ejemplo de `cursos.html`:

```

``html
{% extends 'padre.html' %}

{% block title %}Página de Cursos{% endblock %}

{% block content %}
  <h1>Bienvenido a la página de cursos</h1>
  <p>Aquí encontrarás todos los cursos disponibles.</p>
{% endblock %}

```

...

2. Eliminar Contenido Redundante:

- En los templates hijos, se elimina cualquier código HTML redundante que ya esté presente en el template padre.
 - Se asegura que solo se modifique el contenido específico dentro de los bloques definidos.
-

Ejemplo Completo:

1. Template Padre (`padre.html`):

```
``html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}My Site{% endblock %}</title>
  <link rel="stylesheet" href="{% static 'css/styles.css' %}">
</head>
<body>
  <header>
    <nav class="navbar navbar-light bg-light static-top">
      <div class="container">
        <a class="navbar-brand" href="{% url 'inicio' %}">Inicio</a>
        <a class="navbar-brand" href="{% url 'cursos' %}">Cursos</a>
        <a class="navbar-brand" href="{% url 'profesores' %}">Profesores</a>
        <a class="navbar-brand" href="{% url 'estudiantes' %}">Estudiantes</a>
        <a class="navbar-brand" href="{% url 'entregables' %}">Entregables</a>
        <a class="btn btn-primary" href="#NADAAUN">INICIAR</a>
      </div>
    </nav>
  </header>
  <main>
    {% block content %}
    <!-- Contenido específico de cada página irá aquí -->
    {% endblock %}
  </main>
  <footer>
    <p>&copy; 2024 My Site</p>
  </footer>
</body>
</html>

...

```

2. Template Hijo (`cursos.html`):

```
```html
{% extends 'padre.html' %}

{% block title %}Página de Cursos{% endblock %}

{% block content %}
 <h1>Bienvenido a la página de cursos</h1>
 <p>Aquí encontrarás todos los cursos disponibles.</p>
{% endblock %}

```
```

Navegación entre Templates:

- Para asegurarse de que el menú de navegación funcione correctamente, es importante definir las URLs correspondientes en el archivo `urls.py` de Django.
- Ejemplo de configuración en `urls.py`:

```
```python
from django.urls import path
from AppCoder import views

urlpatterns = [
 path("", views.inicio, name="Inicio"),
 path('cursos/', views.cursos, name="Cursos"),
 path('profesores/', views.profesores, name="Profesores"),
 path('estudiantes/', views.estudiantes, name="Estudiantes"),
 path('entregables/', views.entregables, name="Entregables"),
]

```
```

Beneficios de Navegar entre Templates:

Consistencia:

Mantiene un diseño uniforme en todas las páginas web.

Facilidad de Mantenimiento:

Permite actualizar el menú de navegación en un solo lugar (el template padre) en lugar de hacerlo en cada página individualmente.

Modularidad:

Facilita la creación y gestión de contenido dinámico específico para cada página mientras se mantiene una estructura común.

Con estos pasos, se logra crear un menú de navegación en el template padre y ajustar los templates hijos para utilizar este menú, garantizando una navegación coherente y eficiente en todo el sitio web.

Panel de Administración

El panel de administración de Django es una herramienta poderosa que permite gestionar los datos de tu aplicación web de manera sencilla y eficiente. A continuación, se detallan los pasos para configurar y utilizar el panel de administración, incluyendo la creación de un superusuario y la configuración básica.

Configuración del Panel de Administración

1. Crear un Superusuario:

- El superusuario tiene acceso completo al panel de administración y puede gestionar todos los aspectos de la base de datos.
- Para crear un superusuario, ejecuta el siguiente comando en la terminal:

```
``bash
python manage.py createsuperuser
```

```
``
```

- Se te pedirá que ingreses un nombre de usuario, correo electrónico y contraseña.

2. Registrar Modelos en el Panel de Administración:

- Para gestionar modelos desde el panel de administración, es necesario registrarlos en `admin.py` de la aplicación correspondiente.
- Ejemplo de cómo registrar un modelo:

```
``python
from django.contrib import admin
from .models import Curso

admin.site.register(Curso)
```

...

- Si `admin.py` no existe, crea este archivo en el directorio de la aplicación.

3. Configuración Básica del Panel de Administración:

- Asegúrate de que la aplicación esté incluida en `INSTALLED\_APPS` en `settings.py`:

```
python
INSTALLED_APPS = [
    # ...
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'AppCoder', # Asegúrate de incluir tu aplicación aquí
]
```

...

Acceder al Panel de Administración

1. Iniciar el Servidor de Desarrollo:

- Ejecuta el siguiente comando para iniciar el servidor:

```
bash
python manage.py runserver
```

...

2. Navegar al Panel de Administración:

- Abre un navegador web y ve a <http://127.0.0.1:8000/admin>.
 - Inicia sesión con el superusuario que creaste anteriormente.
-

Gestión de Datos desde el Panel de Administración

1. Agregar y Modificar Datos:

- Una vez que hayas iniciado sesión, verás una interfaz para agregar, modificar y eliminar instancias de los modelos registrados.
- Puedes agregar nuevos registros haciendo clic en "Add" junto al nombre del modelo.

2. Ejemplo de Registro de Modelo (Curso):

- En `models.py` de tu aplicación, define el modelo `Curso`:

```
```python
from django.db import models

class Curso(models.Model):
 nombre = models.CharField(max_length=100)
 camada = models.IntegerField()

 def __str__(self):
 return self.nombre

...`
```

- Registra este modelo en `admin.py`:

```
```python
from django.contrib import admin
from .models import Curso

admin.site.register(Curso)

...`
```

3. Visualización y Administración:

- Después de registrar el modelo, podrás ver y administrar instancias de `Curso` en el panel de administración.
- Podrás agregar, editar y eliminar registros de manera interactiva a través de la interfaz del administrador.

Configuraciones Adicionales:

1. Personalizar la Apariencia del Panel:

- Puedes personalizar la apariencia del panel de administración creando un archivo `admin.py` más detallado.
- Ejemplo de personalización:

```
```python
from django.contrib import admin
from .models import Curso

class CursoAdmin(admin.ModelAdmin):
 list_display = ('nombre', 'camada')
 search_fields = ('nombre',)

admin.site.register(Curso, CursoAdmin)
...`
```

```
...
```

## 2. Añadir Más Modelos:

- Sigue el mismo procedimiento para registrar y administrar otros modelos en tu aplicación.

---

# Ejemplo Completo de Configuración del Panel de Administración:

## 1. Definición del Modelo (`models.py`):

```
```python
from django.db import models

class Curso(models.Model):
    nombre = models.CharField(max_length=100)
    camada = models.IntegerField()

    def __str__(self):
        return self.nombre

...
```
```

## 2. Registro y Personalización en el Administrador (`admin.py`):

```
```python
from django.contrib import admin
from .models import Curso

class CursoAdmin(admin.ModelAdmin):
    list_display = ('nombre', 'camada')
    search_fields = ('nombre',)

admin.site.register(Curso, CursoAdmin)

...
```
```

## 3. Creación de Superusuario:

```
```bash
python manage.py createsuperuser
```
```

'''

#### 4. Iniciar el Servidor y Acceder al Panel de Administración:

```
'''bash
python manage.py runserver
```

'''

- Navega a [`http://127.0.0.1:8000/admin`](http://127.0.0.1:8000/admin) e inicia sesión con el superusuario.

---

## Beneficios del Panel de Administración de Django:

### Eficiencia:

Facilita la gestión de datos sin necesidad de escribir consultas SQL manuales.

### Seguridad:

Permite controlar el acceso a diferentes partes del sistema mediante permisos y usuarios.

### Flexibilidad:

Permite personalizar la interfaz de administración para adaptarse a las necesidades específicas del proyecto.

---

Con estos pasos, podrás configurar y utilizar el panel de administración de Django para gestionar eficientemente los datos de tu aplicación web, proporcionando una interfaz intuitiva y poderosa para administradores y desarrolladores.

---

## Creación de Usuarios

La creación de usuarios en el panel de administración de Django es una tarea fundamental para gestionar quién tiene acceso y qué permisos tiene cada usuario dentro de la aplicación.

### Acceder al Panel de Administración

#### 1. Iniciar el Servidor de Desarrollo:

- Para acceder al panel de administración, primero debes iniciar el servidor de desarrollo de Django.
- Ejecuta el siguiente comando en tu terminal:

```
```bash
python manage.py runserver
```
```

## 2. Navegar al Panel de Administración:

- Abre un navegador web y ve a <http://127.0.0.1:8000/admin>.
- Inicia sesión con un superusuario.

---

## Creación de Nuevos Usuarios

### 1. Ingresar al Panel de Administración:

- Una vez que hayas iniciado sesión en el panel de administración, verás una sección llamada "Usuarios" dentro del grupo "Autenticación y autorización".

### 2. Agregar un Nuevo Usuario:

- Haz clic en "Usuarios".
- Luego, haz clic en "Agregar usuario" (Add user) en la esquina superior derecha.
- Completa el formulario inicial con el nombre de usuario y la contraseña para el nuevo usuario.
- Haz clic en "Guardar y continuar editando" (Save and continue editing).

### 3. Completar el Perfil del Usuario:

- En la siguiente pantalla, completa la información adicional del usuario, como nombre, apellido, correo electrónico, etc.
- Aquí también puedes asignar el estado del usuario, como si el usuario está activo, si es un superusuario, o si tiene permisos de personal (staff).

---

## Asignación de Permisos a los Usuarios

### 1. Definir Permisos del Usuario:

- En la misma pantalla donde completaste el perfil del usuario, desplázate hacia abajo hasta la sección "Permisos".
- Aquí puedes asignar permisos específicos al usuario marcando las casillas correspondientes a los permisos que deseas otorgar.

### 2. Asignar Grupos:

- También puedes agregar el usuario a uno o más grupos.
- Los grupos son una manera conveniente de administrar permisos para múltiples usuarios. Puedes definir un conjunto de permisos para un grupo y luego agregar usuarios a ese grupo para que hereden esos permisos.

- Para agregar el usuario a un grupo, selecciona los grupos deseados en la lista "Grupos".

### 3. Guardar los Cambios:

- Después de asignar los permisos y grupos, haz clic en "Guardar" (Save) en la parte inferior de la página para aplicar los cambios.
- 

## Ejemplo Práctico

### 1. Crear un Nuevo Usuario:

- Supongamos que deseas crear un usuario llamado "Juan Perez" con permisos para ver y modificar datos de cursos.
- Accede al panel de administración y agrega un nuevo usuario con el nombre de usuario "juanperez" y una contraseña segura.

### 2. Completar el Perfil del Usuario:

- Rellena los campos con el nombre "Juan", apellido "Perez", y correo electrónico "[juanperez@example.com](mailto:juanperez@example.com)".

### 3. Asignar Permisos:

- En la sección de permisos, marca las casillas correspondientes a "Can view course" y "Can change course".

### 4. Agregar a un Grupo:

- Si tienes un grupo llamado "Editores de Cursos" con los permisos necesarios, agrega a Juan a este grupo seleccionándolo en la lista de grupos.

### 5. Guardar los Cambios:

- Haz clic en "Guardar" para finalizar la creación del usuario y la asignación de permisos.
- 

## Configuración de Permisos en `admin.py`

- Para gestionar mejor los permisos, puedes personalizar la administración de usuarios en el archivo `admin.py`.
- Ejemplo de personalización:

```
```python
from django.contrib import admin
from django.contrib.auth.models import User
from django.contrib.auth.admin import UserAdmin as BaseUserAdmin

class UserAdmin(BaseUserAdmin):
    list_display = ('username', 'email', 'first_name', 'last_name', 'is_staff')
    search_fields = ('username', 'first_name', 'last_name', 'email')
```

```
readonly_fields = ('date_joined', 'last_login')

fieldsets = (
    (None, {'fields': ('username', 'password')}),
    ('Personal info', {'fields': ('first_name', 'last_name', 'email')}),
    ('Permissions', {'fields': ('is_active', 'is_staff', 'is_superuser', 'groups',
'user_permissions')}),
    ('Important dates', {'fields': ('last_login', 'date_joined')}),
)

admin.site.unregister(User)
admin.site.register(User, UserAdmin)

...
```

Beneficios de la Gestión de Usuarios:

Seguridad:

Controla quién tiene acceso a diferentes partes de la aplicación.

Eficiencia:

Facilita la asignación de permisos y la administración de usuarios.

Flexibilidad:

Permite una gestión detallada y personalizada de los permisos para cada usuario o grupo de usuarios.

Con estos pasos, podrás crear y gestionar usuarios en el panel de administración de Django de manera eficiente, asegurando que cada usuario tenga los permisos adecuados para su rol dentro de la aplicación.

Formularios en Django

Los formularios en Django son una herramienta fundamental para interactuar con los datos de nuestra aplicación web. Permiten a los usuarios enviar información que luego se puede procesar y almacenar en la base de datos. A continuación, se presenta una explicación detallada sobre la creación y manejo de formularios en Django, incluyendo el funcionamiento de los métodos GET y POST.

Creación de Formularios en Django

Paso 1: Definición del Formulario:

- Django proporciona una clase `Form` en el módulo `django.forms` para definir formularios.
- Ejemplo de un formulario básico para agregar un curso (`forms.py`):

```
```python
from django import forms

class CursoFormulario(forms.Form):
 nombre = forms.CharField(max_length=100)
 camada = forms.IntegerField()

...

```

---

## Paso 2. Creación de la Vista:

- En el archivo `views.py`, se define una vista que manejará el formulario.
- Ejemplo de vista para manejar el formulario de curso:

```
```python
from django.shortcuts import render
from .forms import CursoFormulario

def cursoFormulario(request):
    if request.method == 'POST':
        form = CursoFormulario(request.POST)
        if form.is_valid():
            # Procesar los datos del formulario
            nombre = form.cleaned_data['nombre']
            camada = form.cleaned_data['camada']
            # Guardar los datos en la base de datos
            curso = Curso(nombre=nombre, camada=camada)
            curso.save()
            return render(request, 'AppCoder/curso_exito.html')
        else:
            form = CursoFormulario()
    return render(request, 'AppCoder/curso_formulario.html', {'form': form})

...

```

Paso 3. Definición de la URL:

- Es necesario registrar la vista en el archivo `urls.py` para que sea accesible desde una URL.
- Ejemplo de configuración de la URL:

```
```python
from django.urls import path
from . import views

urlpatterns = [
 path('cursoFormulario/', views.cursoFormulario, name='cursoFormulario'),
]

...```
```

---

### Paso 4. Creación del Template:

- Se crea una plantilla HTML que renderizará el formulario.
- Ejemplo de plantilla (`curso\_formulario.html`):

```
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Formulario - Agregar Curso</title>
</head>
<body>
    <h1>Agregar Curso</h1>
    <form action="/cursoFormulario/" method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Enviar</button>
    </form>
</body>
</html>

...```
```

Funcionamiento de los Métodos GET y POST**

1. Método GET:

- El método GET se utiliza para solicitar datos de un servidor.
- En el contexto de formularios, GET se usa principalmente para recuperar datos.
- Ejemplo de un formulario que usa GET:

```
```html
<form action="/buscarCurso/" method="get">
 <label for="camada">Camada:</label>
 <input type="text" id="camada" name="camada">
 <button type="submit">Buscar</button>
</form>

```
```

2. Método POST:

- El método POST se utiliza para enviar datos al servidor.
- En el contexto de formularios, POST se usa para enviar datos que se deben procesar, como agregar o modificar registros en la base de datos.
- Ejemplo de un formulario que usa POST:

```
```html
<form action="/cursoFormulario/" method="post">
 {% csrf_token %}
 {{ form.as_p }}
 <button type="submit">Enviar</button>
</form>

```
```

Procesamiento de Datos en la Vista

1. Manejo del Método POST:

- En la vista, se verifica si la solicitud es de tipo POST y se procesa el formulario.
- Ejemplo de manejo del método POST en la vista:

```
```python
if request.method == 'POST':
 form = CursoFormulario(request.POST)
 if form.is_valid():
 nombre = form.cleaned_data['nombre']

```
```

```
        camada = form.cleaned_data['camada']
        # Guardar los datos en la base de datos
        curso = Curso(nombre=nombre, camada=camada)
        curso.save()
        return render(request, 'AppCoder/curso_exito.html')

    ...
```

2. Manejo del Método GET:

- Si la solicitud es de tipo GET, simplemente se renderiza el formulario vacío.
- Ejemplo de manejo del método GET en la vista:

```
    ``python
    else:
        form = CursoFormulario()
        return render(request, 'AppCoder/curso_formulario.html', {'form': form})

    ...
```

Validación de Formularios

- Django proporciona mecanismos integrados para la validación de formularios.
 - Al llamar a `form.is_valid()`, Django verifica que todos los datos del formulario sean válidos según las reglas definidas en el formulario.
 - Los datos limpios se pueden acceder mediante `form.cleaned_data`.
-

Ejemplo Completo de Formulario

Paso 1. Definición del Formulario (`forms.py`):

```
``python
from django import forms

class CursoFormulario(forms.Form):
    nombre = forms.CharField(max_length=100)
    camada = forms.IntegerField()

    ...
```

Paso 2. Definición de la Vista (`views.py`):

```

```python
from django.shortcuts import render
from .forms import CursoFormulario

def cursoFormulario(request):
 if request.method == 'POST':
 form = CursoFormulario(request.POST)
 if form.is_valid():
 nombre = form.cleaned_data['nombre']
 camada = form.cleaned_data['camada']
 # Guardar los datos en la base de datos
 curso = Curso(nombre=nombre, camada=camada)
 curso.save()
 return render(request, 'AppCoder/curso_exito.html')
 else:
 form = CursoFormulario()
 return render(request, 'AppCoder/curso_formulario.html', {'form': form})
 ...

```

---

### Paso 3. Definición de la URL (`urls.py`):

```

```python
from django.urls import path
from . import views

urlpatterns = [
    path('cursoFormulario/', views.cursoFormulario, name='cursoFormulario'),
]
...

```

Paso 4. Creación del Template (`curso\_formulario.html`):

```

```html
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Formulario - Agregar Curso</title>
</head>
<body>
 <h1>Agregar Curso</h1>
 <form action="/cursoFormulario/" method="post">
 {% csrf_token %}
 {{ form.as_p }}
 <button type="submit">Enviar</button>
 </form>

```

```
</form>
</body>
</html>
...
```

Con estos pasos, podrás crear y manejar formularios en Django de manera eficiente, permitiendo a los usuarios interactuar con los datos de tu aplicación web de manera sencilla y segura.

---

## Búsqueda con Formularios

Los formularios de búsqueda en Django permiten a los usuarios consultar datos almacenados en la base de datos de manera eficiente. A continuación, se presenta una guía detallada sobre cómo crear y manejar formularios de búsqueda en Django.

### Creación de Formularios de Búsqueda en Django

#### Paso 1. Definición del Formulario de Búsqueda:

- Primero, define un formulario que recibirá los parámetros de búsqueda.
- Ejemplo de un formulario de búsqueda de cursos (`forms.py`):

```
```python
from django import forms

class BusquedaCursoFormulario(forms.Form):
    camada = forms.IntegerField(label='Camada')

...
```
```

#### Paso 2. Creación de la Vista para Manejar el Formulario de Búsqueda:

- Define una vista que maneje tanto las solicitudes GET como POST para el formulario de búsqueda.
- Ejemplo de vista para manejar el formulario de búsqueda de cursos (`views.py`):

```
```python
from django.shortcuts import render
from .forms import BusquedaCursoFormulario
from .models import Curso

def buscarCurso(request):
    if request.method == 'GET':
        form = BusquedaCursoFormulario(request.GET)
...
```
```

```

 if form.is_valid():
 camada = form.cleaned_data['camada']
 resultados = Curso.objects.filter(camada=camada)
 return render(request, 'AppCoder/resultados_busqueda.html', {'resultados':
resultados, 'form': form})
 else:
 form = BusquedaCursoFormulario()
 return render(request, 'AppCoder/buscar_curso.html', {'form': form})

...

```

---

### Paso 3. Definición de la URL para la Búsqueda:

- Registra la vista en el archivo `urls.py` para que sea accesible desde una URL.
- Ejemplo de configuración de la URL (`urls.py`):

```

'''python
from django.urls import path
from . import views

urlpatterns = [
 path('buscarCurso/', views.buscarCurso, name='buscarCurso'),
]

...

```

### Paso 4. Creación del Template para el Formulario de Búsqueda:

- Crea una plantilla HTML que renderice el formulario de búsqueda.
- Ejemplo de plantilla (`buscar\_curso.html`):

```

'''html
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Buscar Curso</title>
</head>
<body>
 <h1>Buscar Curso por Camada</h1>
 <form method="get" action="{% url 'buscarCurso' %}">
 {{ form.as_p }}
 <button type="submit">Buscar</button>
 </form>
</body>
</html>

...

```

---

### Paso 5. Creación del Template para Mostrar Resultados:

- Crea una plantilla HTML que muestre los resultados de la búsqueda.
- Ejemplo de plantilla (`resultados\_busqueda.html`):

```
``html
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Resultados de Búsqueda</title>
</head>
<body>
 <h1>Resultados de Búsqueda</h1>
 {% if resultados %}

 {% for curso in resultados %}
 {{ curso.nombre }} - Camada: {{ curso.camada }}
 {% endfor %}

 {% else %}
 <p>No se encontraron resultados para la camada especificada.</p>
 {% endif %}
 Nueva Búsqueda
</body>
</html>

``
```

---

## Funcionamiento del Formulario de Búsqueda

### 1. Método GET para Búsqueda:

- El método GET se utiliza para solicitar datos del servidor. En el contexto de formularios de búsqueda, GET se usa para enviar los parámetros de búsqueda.
- Ejemplo de formulario que usa GET:

```
``html
<form method="get" action="{% url 'buscarCurso' %}">
 {{ form.as_p }}
 <button type="submit">Buscar</button>
</form>

``
```

### 2. Procesamiento de Datos en la Vista:

- La vista verifica si la solicitud es de tipo GET y procesa el formulario.
- Ejemplo de manejo del método GET en la vista (`views.py`):

```
```python
if request.method == 'GET':
    form = BusquedaCursoFormulario(request.GET)
    if form.is_valid():
        camada = form.cleaned_data['camada']
        resultados = Curso.objects.filter(camada=camada)
        return render(request, 'AppCoder/resultados_busqueda.html', {'resultados':
resultados, 'form': form})
    else:
        form = BusquedaCursoFormulario()
        return render(request, 'AppCoder/buscar_curso.html', {'form': form})

...
```
```

---

## Ejemplo Completo de Búsqueda con Formularios

### Paso 1. Definición del Formulario (`forms.py`):

```
```python
from django import forms

class BusquedaCursoFormulario(forms.Form):
    camada = forms.IntegerField(label='Camada')

...
```
```

### Paso 2. Definición de la Vista (`views.py`):

```
```python
from django.shortcuts import render
from .forms import BusquedaCursoFormulario
from .models import Curso

def buscarCurso(request):
    if request.method == 'GET':
        form = BusquedaCursoFormulario(request.GET)
        if form.is_valid():
            camada = form.cleaned_data['camada']
            resultados = Curso.objects.filter(camada=camada)
```
```

```

 return render(request, 'AppCoder/resultados_busqueda.html', {'resultados':
resultados, 'form': form})
 else:
 form = BusquedaCursoFormulario()
 return render(request, 'AppCoder/buscar_curso.html', {'form': form})

'''

```

---

### Paso 3. Definición de la URL (`urls.py`):

```

'''python
from django.urls import path
from . import views

urlpatterns = [
 path('buscarCurso/', views.buscarCurso, name='buscarCurso'),
]

'''

```

### Paso 4. Creación del Template para el Formulario de Búsqueda (`buscar\_curso.html`):

```

'''html
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Buscar Curso</title>
</head>
<body>
 <h1>Buscar Curso por Camada</h1>
 <form method="get" action="{% url 'buscarCurso' %}">
 {{ form.as_p }}
 <button type="submit">Buscar</button>
 </form>
</body>
</html>

'''

```

---

### Paso 5. Creación del Template para Mostrar Resultados (`resultados\_busqueda.html`):

```

'''html
<!DOCTYPE html>
<html lang="en">
<head>

```

```
<meta charset="UTF-8">
<title>Resultados de Búsqueda</title>
</head>
<body>
 <h1>Resultados de Búsqueda</h1>
 {% if resultados %}

 {% for curso in resultados %}
 {{ curso.nombre }} - Camada: {{ curso.camada }}
 {% endfor %}

 {% else %}
 <p>No se encontraron resultados para la camada especificada.</p>
 {% endif %}
 Nueva Búsqueda
</body>
</html>
```

---

Con estos pasos, podrás crear y manejar formularios de búsqueda en Django, permitiendo a los usuarios consultar datos en la base de datos de manera eficiente y obtener los resultados deseados.

---

## Ordenamiento de Templates

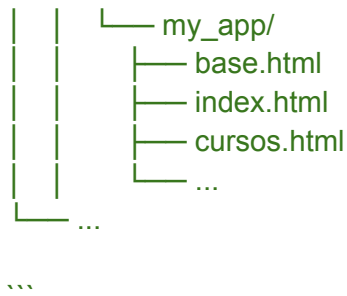
Organizar los templates en un proyecto Django de manera clara y eficiente es crucial para mantener la escalabilidad y facilidad de mantenimiento del proyecto. La herencia de templates es una técnica poderosa que permite crear una estructura coherente y modular para los archivos HTML. A continuación, se presenta una guía detallada sobre cómo organizar los templates en un proyecto Django utilizando la herencia de templates.

## Estructura de Directorios de Templates

### 1. Directorio de Templates:

- Se recomienda crear un directorio de templates dentro de cada aplicación del proyecto Django para mantener los archivos organizados.
- Estructura de ejemplo:

```
...
my_project/
|— my_app/
| |— templates/
```



## 2. Template Base:

- El template base (padre) contiene elementos comunes a todas las páginas, como la barra de navegación, el pie de página y el encabezado.
- Ejemplo de `base.html`:

```
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}My Site{% endblock %}</title>
  <link rel="stylesheet" href="{% static 'css/styles.css' %}">
</head>
<body>
  <header>
    <nav class="navbar navbar-light bg-light static-top">
      <div class="container">
        <a class="navbar-brand" href="{% url 'inicio' %}">Inicio</a>
        <a class="navbar-brand" href="{% url 'cursos' %}">Cursos</a>
        <a class="navbar-brand" href="{% url 'profesores' %}">Profesores</a>
        <a class="navbar-brand" href="{% url 'estudiantes' %}">Estudiantes</a>
        <a class="navbar-brand" href="{% url 'entregables' %}">Entregables</a>
        <a class="btn btn-primary" href="#NADAAUN">INICIAR</a>
      </div>
    </nav>
  </header>
  <main>
    {% block content %}
    <!-- Contenido específico de cada página irá aquí -->
    {% endblock %}
  </main>
  <footer>
    <p>&copy; 2024 My Site</p>
  </footer>
</body>
</html>
```

...

Herencia de Templates

1. Extender el Template Base:

- Los templates hijos extienden el template base utilizando la directiva `{% extends 'my\_app/base.html' %}`.
- Ejemplo de `index.html`:

```
```html
{% extends 'my_app/base.html' %}

{% block title %}Inicio{% endblock %}

{% block content %}
 <h1>Bienvenido a My Site</h1>
 <p>Este es el contenido de la página de inicio.</p>
{% endblock %}
```
```

2. Definir Bloques en el Template Base:

- Los bloques definidos en el template base permiten a los templates hijos insertar contenido específico.
- Ejemplo de bloques en `base.html`:

```
```html
<title>{% block title %}My Site{% endblock %}</title>
<main>
 {% block content %}
 <!-- Contenido específico de cada página irá aquí -->
 {% endblock %}
</main>
```
```

Ejemplo Completo de Organización de Templates

1. Template Base (`base.html`):

```
``html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}My Site{% endblock %}</title>
  <link rel="stylesheet" href="{% static 'css/styles.css' %}">
</head>
<body>
  <header>
    <nav class="navbar navbar-light bg-light static-top">
      <div class="container">
        <a class="navbar-brand" href="{% url 'inicio' %}">Inicio</a>
        <a class="navbar-brand" href="{% url 'cursos' %}">Cursos</a>
        <a class="navbar-brand" href="{% url 'profesores' %}">Profesores</a>
        <a class="navbar-brand" href="{% url 'estudiantes' %}">Estudiantes</a>
        <a class="navbar-brand" href="{% url 'entregables' %}">Entregables</a>
        <a class="btn btn-primary" href="#NADAAUN">INICIAR</a>
      </div>
    </nav>
  </header>
  <main>
    {% block content %}
    <!-- Contenido específico de cada página irá aquí -->
    {% endblock %}
  </main>
  <footer>
    <p>&copy; 2024 My Site</p>
  </footer>
</body>
</html>

...

```

2. Template Hijo (`index.html`):

```
``html
{% extends 'my_app/base.html' %}

{% block title %}Inicio{% endblock %}

{% block content %}
  <h1>Bienvenido a My Site</h1>
  <p>Este es el contenido de la página de inicio.</p>
{% endblock %}

```

...

3. Otro Template Hijo (`cursos.html`):

```
``html
{% extends 'my_app/base.html' %}

{% block title %}Cursos{% endblock %}

{% block content %}
  <h1>Cursos Disponibles</h1>
  <p>Aquí encontrarás todos los cursos que ofrecemos.</p>
{% endblock %}
``
```

Beneficios de Organizar Templates con Herencia

Reutilización de Código:

La herencia de templates permite reutilizar componentes comunes (como encabezados y pies de página) en múltiples páginas.

Mantenimiento Simplificado:

Al actualizar elementos comunes en el template base, los cambios se reflejan automáticamente en todas las páginas que extienden dicho template.

Estructura Coherente:

Mantiene una estructura uniforme y coherente en todas las páginas del sitio web, mejorando la experiencia del usuario.

Consejos para la Organización de Templates

Nombres Descriptivos:

Utiliza nombres descriptivos para los archivos de templates para que sea fácil identificar su propósito.

Subdirectorios:

Considera utilizar subdirectorios dentro del directorio de templates para organizar mejor los archivos relacionados con diferentes funcionalidades o secciones del sitio.

Componentes Reutilizables:

Crea componentes reutilizables (como menús de navegación, formularios, etc.) y guárdalos en archivos de templates separados que se puedan incluir en otros templates usando `{% include 'ruta/al/componente.html' %}`.

Con estos pasos, podrás organizar los templates en tu proyecto Django de manera clara y eficiente, utilizando la herencia de templates para mantener una estructura modular y fácil de mantener.

Material complementario:

Recursos complementarios

¿Quieres saber más?

[Código de registro](#)